

COMPLETE PROJECT & CYBERSECURITY

AUDIT: Project-Ticket

Scope: D:\Project-ticket\Project-Ticket · **Date:** 2026-08-29 · **Method:** We reviewed the actual source code and configuration files, and re-checked the fixes made earlier today. We did not attack the live system or run any destructive tests.

This report reflects the codebase after the fixes applied earlier today (a security-headers tool called `helmet`, a setting called `trust proxy`, general rate limiting to slow down abuse, a minimum password length rule, trimming what the `/healthz` health-check page reveals, and cleaning up leftover files). See section 51 for the full list of what changed. This report replaces the security picture described in the earlier `docs/DEEP_AUDIT_REPORT.md`, though that older report is still useful for background on the architecture and how data flows through the system.

A note on names: the account with the highest level of access in the code is called the `superadmin` role. In this document we call that person the "Founder" instead, since that is the plain-language title people actually use. There is also a separate, lower-level `founder` role in the code, and we call that out by its exact name whenever the difference matters. File and function names in the code (like `superAdminController.js`) are left exactly as they are, since those are literal references to real files.

1. Executive Summary

Project-Ticket's security setup is unusually consistent for a project this size. There is one API (the backend service that the website talks to), one place data is stored, and every important access decision (who you are, what role you have, whether you own the thing you're trying to change) is checked on the server, not just hidden in the website's design. We checked this down to the level of individual code files, across all ten backend controllers (the pieces of code that handle each type of request). Two open questions from the previous audit were resolved this time: whether an employee could change another employee's task status, and whether the dashboard/reporting pages could leak information across departments. Both were checked thoroughly and confirmed safe, not gaps.

What was genuinely missing has now been fixed. There were no security-related response headers (small pieces of information the server sends back that tell the browser how to behave safely); a tool called `helmet()` now adds those. Rate limiting (a feature that slows down someone hammering the server with requests) only covered the login page before; it is now applied to every page. Rate limiting was also silently broken behind the hosting provider's proxy (a kind of middleman server), because a setting called `trust proxy` was missing, meaning every visitor could accidentally share one "bucket" and get blocked together; that is fixed. The password-length rule used to be inconsistent; it's

now the same everywhere a password is set, at least 10 characters. And the health-check page (/healthz , used to confirm the server is alive) used to leak some internal details to anyone who visited it; that's been trimmed down.

SECURITY SCORE: 79 out of 100 (see section 48 for how each category was scored and how this number was calculated).

The single biggest thing still missing is not a code problem, it's a process problem: there is no automated pipeline (commonly called CI/CD, short for Continuous Integration/Continuous Deployment) that checks for security or dependency problems before new code goes live. There's also no multi-factor authentication (MFA, an extra login step like a text-message code, on top of a password). And there's no organized, searchable logging system for ordinary day-to-day traffic (admin actions and failed logins are the exception, those are already reliably recorded). None of these three things are actual holes that someone could exploit today; they're the honest list of what to harden next.

2. Project Overview

Project-Ticket is Fute Services' internal tool for running day-to-day operations: tracking HR and IT complaints, managing leave requests and approvals, keeping an inventory of IT equipment, an HR desk module (covering employees, job candidates, interviews, and attendance), coordinator tasks and projects, and a tracker for production renders (video/graphics jobs). It serves seven types of user accounts: founder , superadmin (which we call "Founder" throughout this document), hr , it , coordinator , employee , plus a few demo-only roles that aren't connected to any real backend logic yet.

3. Complete Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE	SECURITY RELEVANCE
Frontend (what you see in the browser)	React 18.2 + Vite 5	React 18.2, Vite 5	The website itself	React automatically escapes text so malicious code typed into a form can't run in the browser (this is the main defense against a type of attack called XSS, explained in section 21)
Backend (the server that handles requests)	Express	^4.18.2	The API the website talks to	Now protected with helmet and rate limiting
Login system	Firebase Auth + jsonwebtoken	firebase-admin ^12.0.0, jsonwebtoken ^9.0.2	Stores accounts and issues login tokens	Password checking is fully handled by Firebase, not by our own code
Database	Firestore	via firebase-admin	Where all the app's data lives	Only the backend server can touch it directly; there's no path for the browser to reach it on its own
File storage	Firebase Storage	via firebase-admin	HR documents	File type and size are checked before anything is accepted
Email	Nodemailer (SMTP)	^6.9.7	Sends notification emails	Any text inserted into an email is escaped so it can't be used to inject fake links or formatting
Security headers	helmet	^8.3.0 (added today)	Adds browser-safety instructions to every response	New
Rate limiting	express-rate-limit	^8.6.2	Slows down abuse	Now applied to every page, not just login
Hosting	Vercel	:	Frontend and backend run as two separate projects	The connection is encrypted (HTTPS) right at Vercel's edge servers
Automated deployment checks (CI/CD)	none	:	:	Confirmed missing, see section 35
Testing	Playwright	end-to-end	Automated browser tests for the frontend	:

4. Complete Project Structure

See `docs/DEEP_AUDIT_REPORT.md` section 4 for the full folder layout. It's unchanged except that two leftover, unused files (a stray `package-lock.json` and an old Java installer folder called `.tools/jdk-21.../`) were deleted today with the owner's approval.

5. File & Folder Responsibilities

FILE/FOLDER	PURPOSE	SECURITY RELEVANCE
<code>main/backend/server.js</code>	The starting point of the backend: sets up the middleware chain (the sequence of checks every request passes through) and the error handler	Now where helmet, the trust-proxy fix, and the general rate limiter live
<code>main/backend/config/firebase.js</code>	Sets up the connection to Firebase (with a fallback to a local test version when real credentials aren't configured)	Holds the highest-trust credentials in the whole app, for the database, login system, and file storage
<code>main/backend/middleware/authMiddleware.js</code>	Checks that a login token is valid, that the session hasn't been signed out remotely, and that the account isn't disabled	The main gatekeeper for every protected page
<code>main/backend/middleware/roleMiddleware.js</code>	Checks whether your account's role is allowed on a given page at all	The main coarse-grained permission check
<code>main/backend/middleware/permissionMiddleware.js</code>	A more detailed, per-action permission check stored in the database (used for IT	A second, finer layer of permission checking

FILE/FOLDER	PURPOSE	SECURITY RELEVANCE
	equipment records)	
<code>main/backend/controllers/authController.js</code>	Handles registering, logging in, logging out, fetching your own profile, and verifying your password	Now requires at least 10 characters when you register
<code>main/backend/controllers/superAdminUserController.js</code>	Lets an admin create, edit, or reset accounts	Now also requires at least 10 characters for admin-created accounts
<code>main/backend/controllers/complaintControllerFactory.js</code>	Shared logic for creating, reading, updating, and deleting HR and IT complaints	This is where the check "do you actually own this complaint" lives, for edits and deletes
<code>main/backend/controllers/taskProjectController.js</code>	Handles tasks and projects	Confirmed: it checks ownership before letting someone update a task's status
<code>main/backend/controllers/securityController.js</code>	Powers the admin's Security console (viewing sessions, unlocking accounts)	Confirmed: every action here is restricted to the Founder role at the routing level
<code>main/backend/utils/sessions.js</code>	Creates and revokes login sessions	Supports the ability to remotely sign a login token out
<code>main/backend/utils/auditLog.js</code>	Records a trail of admin actions	Writes to the <code>audit_logs</code> collection in the database
<code>main/backend/utils/upload.js</code>	Configures file uploads (using a	Only allows certain file types, and caps

FILE/FOLDER	PURPOSE	SECURITY RELEVANCE
	library called Multer)	size at 10MB, keeping files in memory rather than writing them to disk
main/backend/utils/mailer.js	Sends notification emails	Escapes any text inserted into an email so it can't inject malicious content

6. Overall Architecture

In plain terms: your browser sends a request over an encrypted connection with a login token attached. That request passes through several safety checks (helmet's headers, rate limiting, an allow-list of trusted websites, and JSON parsing) before it reaches the actual login check, the role check, the fine-grained permission check, and finally the code that does the real work and checks you own whatever you're touching. From there it may talk to the database, the login system, file storage, or the email server. If anything goes wrong, one central error handler sends back a safe, generic message to you while logging the full detail only on the server.

```

Browser (the website)
|   Encrypted connection (HTTPS) with a login token attached
v
Backend server: security headers, then rate limiting (300 requests per 15 min),
then a check for trusted websites, then reading the incoming data
|
|-- login check -> role check -> fine-grained permission check -> the actual
|   page logic (which also checks you own what you're editing)
|
|       |
|       |--> Database (all app data, reachable only by the backend server)
|       |--> Login system (checks your password)
|       |--> File storage (HR documents)
|       `--> Email server (sends notifications)
|
|-- central error handler: sends you a safe generic message, logs the full
    detail only on the server

```

7. Frontend Architecture

The website uses React's "Context" feature to share data between different parts of the app, and it never talks to the database directly, everything goes through the backend API. We searched for

common ways that malicious code could sneak into a page (a few specific patterns called `dangerouslySetInnerHTML`, `eval`, `new Function`, and unchecked `postMessage` listeners) and found none of them anywhere in the frontend code. We re-checked this fresh in this pass rather than just trusting the previous audit's finding.

8. Backend Architecture

Every request goes through the same sequence: the login check, then the role check, then the fine-grained permission check where relevant, then the actual page logic, then (when the data needs to stay consistent) a database transaction, then the response. See section 5 above for exactly what changed today.

9. Database Architecture

The app only uses Firestore (Google's database). We confirmed this directly: two other database-related packages (`@supabase` and `@google-cloud`) do exist in the project's installed dependencies, but they're only there because Firestore's own tools depend on them internally, the app's own code never uses them. There's no database-level "security rules" file, which sounds concerning but is actually fine here, because the browser has no direct path to the database at all; every request has to go through the backend server first. See section 26 for more detail.

10. API Architecture

The backend exposes roughly 60 different endpoints (specific URLs the website can call) across 10 route files. A full table of exactly who can call what is in `docs/DEEP_AUDIT_REPORT.md`, section 9. Today's fixes didn't change any of those permission rules, they were changes to the safety net around the whole API, not to individual pages.

11. Complete Request Lifecycle

Here's what happens, step by step, when you click something in the app that needs data from the server:

```
You click something -> the relevant piece of the page -> the app's shared
data/state -> the API helper file (api.js) -> an HTTP request is sent
  -> passes through helmet / rate limiting / trusted-website check
  -> login check -> role check -> fine-grained permission check
  -> the actual page logic (checks the data is valid, and that you own it)
  -> the database
  -> a response comes back -> the app's shared data updates -> the screen redraws
```

12. Complete Data Flow

As a concrete example: filing an HR complaint, and then having it resolved, is documented step by step with exact file names and line numbers in `docs/DEEP_AUDIT_REPORT.md`, section 10. Today's fixes didn't touch that logic, only the safety net around the whole system.

13. Data Inventory

DATA	WHERE IT COMES FROM	HOW IT'S HANDLED	WHERE IT'S STORED	SENSITIVE?
Login credentials	The login/signup form	Checked through Firebase's login system	Firebase's own login system (never this app's own database)	Yes
Login tokens	Issued when you log in	A signed token (like a temporary digital ID card)	Kept in the browser	Yes
Complaints	HR/IT ticket forms	Tagged with who owns it, saved as a single all-or-nothing database write	The <code>hr_complaints</code> / <code>it_complaints</code> collections	Yes (this is personal grievance content)
Employee documents	HR desk uploads	File type and size checked first	Firebase file storage	Yes
Audit trail	Admin actions	Recorded automatically	The <code>audit_logs</code> collection	Yes (internal record-keeping)
Failed login attempts	Login attempts	Recorded with the visitor's IP address	The <code>failed_logins</code> collection	Yes (used for security monitoring)

14. Sensitive Data Flow

See `docs/DEEP_AUDIT_REPORT.md` , section 15, this is unchanged from that report.

15. Authentication Audit

Here's what happens step by step when someone logs in:

```
You type your email/password -> the login form -> the app (AuthContext) sends  
a request -> the server checks the fields are filled in -> Firebase's login  
system verifies the password -> the server checks you're not locked out and  
records a failed attempt if the password was wrong -> a login token is issued  
(valid for 7 days, with a session ID embedded in it) -> the token is stored  
in the browser -> you're logged in
```

QUESTION	ANSWER	WHERE THIS IS PROVEN IN THE CODE
How are passwords protected?	Fully handled by Firebase's login system (using a strong hashing method called scrypt), our own code never touches raw passwords	There's no password-hashing code in this codebase at all, and that's correct, not a gap: Firebase already does it properly
What kind of login token is used?	A signed token (JWT, using an algorithm called HS256) signed with a secret key called <code>JWT_SECRET</code>	<code>authController.js</code>
How long does a login token last?	7 days	<code>authController.js</code> , lines 55 to 59
Can a token be refreshed without logging in again?	No, but there's a workaround: every request re-checks whether the session was signed out remotely, so a revoked login stops working almost immediately even though the token itself doesn't expire for 7 days	<code>authMiddleware.js</code>
Admin-driven password reset	Requires at least 10 characters, and is recorded in the audit trail	<code>superAdminUserController.js</code> , lines 268 to 289
Password rule for self-signup	At least 10 characters, fixed today, this used to not be enforced at all	<code>authController.js</code>
Password rule for admin-created accounts	At least 10 characters, fixed today, this used to not be enforced either	<code>superAdminUserController.js</code>
Protection against repeated password guessing	Account locks after 5 wrong attempts, plus a limit of 10 login attempts per 15 minutes from the same visitor	<code>authController.js</code> , <code>authRoutes.js</code>
Multi-factor authentication (a second login step, like a text code)	Not found anywhere in the code we reviewed	:

16. Authorization Audit

The app uses role-based access control (checking what your account type is allowed to do), plus resource-level permissions for IT equipment, plus ownership checks built into the individual page-handling code. The critical question we asked was: can a user get at someone else's data just by changing an ID number in a request? We checked every place data gets changed, and the answer is no. The two previously open questions from the last audit are now resolved:

- Updating a task's status (`PATCH /tasks/:id/status`) has no role check at the routing level, but we confirmed it's still safe: the code in `taskProjectController.js` (lines 65 to 83) checks that you're either a coordinator/Founder, or that the task is actually assigned to you, and rejects everyone else with a "forbidden" response.

- The dashboard and reporting pages are also confirmed safe: every one of those pages is restricted to the Founder role at the routing level, so there's no partial-access path where a lower-level role could sneak in.

One thing worth knowing about (called `searchByToken`, in `complaintControllerFactory.js`, lines 191 to 198): any logged-in user can look up any complaint if they know its 6-character token. This is intentional, tokens work like a shared tracking number, and there are about 2.2 billion possible combinations, so guessing one isn't realistic. It's a deliberate design choice, not a bug, and we've rated it LOW risk.

17. Session Security

PROPERTY	VALUE
What identifies your session	A login token plus a matching record in the <code>sessions</code> database collection, linked by a session ID
Cookies	Not used at all, the login token is sent in a header instead, which also means a common attack called CSRF doesn't apply here (see section 22)
Where the token is stored	In the browser's storage, which JavaScript on the page can technically read (there's no extra browser-level protection here, because that protection only applies to cookies, which this app doesn't use)
How long it lasts	7 days, but it can be revoked and take effect within about 60 seconds
Does the token get replaced periodically?	No, one token lasts from login until it expires or is revoked
Can a login be revoked?	Yes, confirmed working: both logging out yourself, and an admin force-logging someone out, flip a "revoked" flag that's checked on every single request
Multiple devices at once	Not restricted, you can be logged in on several devices at the same time; the Founder can see all of them and revoke any one individually

18. API Security

We found no broken login checks. We found no cases where someone could access or change data that isn't theirs (see section 16). We found no way to sneak extra fields into a request that shouldn't be settable, for example your account role can never be changed by sending it in a request body; it's protected by a hardcoded allow-list (`superAdminController.js`, lines 144 to 145). We didn't specifically test for "parameter pollution" (sending the same field twice to confuse the server), this relies on Express's default behavior. Rate limiting is now applied everywhere, not just the login page (fixed today). The list of trusted websites allowed to call the API is a specific allow-list, not a wildcard that lets anyone in (see section 23).

19. OWASP Top 10 Assessment

(OWASP is a well-known industry list of the ten most common categories of web security problems. This table checks the app against each one.)

CATEGORY	STATUS	EVIDENCE	SEVERITY	RECOMMENDATION
A01: Broken access control	Adequate	Every role and ownership check happens on the server, confirmed across all 10 controllers	:	Keep applying the same pattern to any new page
A02: Cryptographic failures	Adequate	Password hashing is fully handled by Firebase; login tokens are signed on the server; the connection is encrypted at Vercel's edge	:	Double-check that encryption isn't only assumed at the platform level (see section 37)
A03: Injection (malicious code sneaking into a database query or command)	Strong	The database's own query builder is used everywhere, with no queries built by pasting text together, and no use of code-execution functions	:	:
A04: Insecure design	Adequate	Session revocation, account lockout, and ownership checks were clearly designed in on purpose, not bolted on as an afterthought	:	:
A05: Security misconfiguration	Fixed today	Security headers added, the trust-proxy setting fixed, the health-check page trimmed	LOW (after the fix)	Was HIGH before the fix
A06: Vulnerable or outdated components	Unknown / partial	Running a dependency-checking tool on the backend reports 9 known issues (8 moderate, 1 high) in indirect dependencies, not yet resolved	MEDIUM	Run the automatic fixer, and carefully review any fix that requires forcing a bigger version jump
A07: Identification and authentication failures	Adequate	Account lockout, rate limiting, and session revocation are all present; there's no multi-factor authentication	MEDIUM (no MFA)	Consider adding MFA specifically for the Founder role
A08: Software and data integrity failures	Adequate	No unsafe deserialization found; there's no automated pipeline to verify what's actually being deployed	MEDIUM	Add an automated pipeline that installs from a locked, verified dependency list
A09: Security logging and monitoring failures	Partial	Admin actions and failed logins are reliably recorded; ordinary request and error logs are	MEDIUM	Add organized, searchable logging (see sections 36 and 46)

CATEGORY	STATUS	EVIDENCE	SEVERITY	RECOMMENDATION
		console-only, not searchable		
A10: Server-side request forgery (tricking the server into making requests on an attacker's behalf)	Not applicable, clean	No server-side requests to attacker-controlled addresses were found (see section 29)	:	:

20. Injection Assessment

There's no traditional SQL database in use, so SQL injection doesn't apply. For the NoSQL database (Firestore) that is used, all queries go through its official query-building tools with fixed field names, we found no dynamically built collection names or queries pasted together from text. We found no command injection risk (we searched the entire backend for code that runs shell commands, and found none). There's no server-side template engine in use (the API only returns plain data, not rendered HTML), so template injection doesn't apply. LDAP injection (a directory-service attack) doesn't apply, this app doesn't use LDAP. We found no expression-injection risk either.

21. XSS Assessment

XSS (cross-site scripting) is when an attacker sneaks malicious code into a page that then runs in another user's browser. We checked three flavors:

Stored XSS (malicious content saved in the database, then shown to other people later): complaint and HR text is saved to the database exactly as typed, but the website displays it through React, which automatically escapes text so it can't run as code. We found no place where that protection is bypassed.

Reflected XSS (malicious content in a URL that gets echoed back into the page): the API only ever returns plain data (JSON), never rendered HTML, so this style of attack doesn't apply to the API itself.

DOM XSS (malicious code injected directly into the page's structure): we searched the entire frontend for the specific patterns that would allow this (`dangerouslySetInnerHTML` , `innerHTML` , `eval` , `new Function`) and found zero, re-checked fresh in this pass.

Overall verdict: we found no way to inject malicious code into the page.

22. CSRF Assessment

CSRF (cross-site request forgery) is an attack where a malicious website tricks your browser into automatically sending a request to a site you're logged into, taking advantage of the browser automatically attaching your login cookie. This app doesn't use cookies at all, your login token is sent in a header instead, and there's no way for another website to make your browser automatically attach that header. So this category of attack simply doesn't apply here, and no extra CSRF protection is needed.

23. CORS Assessment

CORS (Cross-Origin Resource Sharing) is the browser rule that controls which websites are allowed to make requests to this API from JavaScript. Here's the actual rule from the code:

```
const isLocalhost = (origin) => /^http:\/\/localhost:\d+$/.test(origin);
app.use(cors({
  origin(origin, callback) {
    if (!origin || allowedOrigins.includes(origin) || isLocalhost(origin)) return callback(null, true);
    callback(new Error('Not allowed by CORS'));
  },
  maxAge: 600,
}));
```

The list of trusted websites (`allowedOrigins`) comes from an environment setting plus one hardcoded production web address. The rule that allows local development traffic is written carefully (it has to match the whole address exactly, from start to end), and we re-checked this pass that it can't be tricked, for example a fake address like `evil.com?localhost:1` correctly fails the check. Requests with no website address attached at all are allowed through, which is expected and normal for things like command-line tools or one server talking to another (CORS doesn't apply to those anyway). There's no wildcard that lets any website in, and no combination of a wildcard with credentials, which would be a real weakness. **Verdict: this is set up correctly.**

24. Security Headers

Before today: there were none at all. Security headers are small pieces of information a server sends back that tell the browser how to behave more safely. The `helmet()` tool is now applied to every response, and we confirmed live (by directly checking a running server's response) that it adds: a Content-Security-Policy (restricts what a page is allowed to load), Strict-Transport-Security (forces the browser to always use an encrypted connection, for one year, including on subdomains), a rule stopping browsers from guessing file types in unsafe ways, a rule stopping the site from being

embedded in another site's frame (a clickjacking protection), a stricter referrer policy, and a couple of others.

25. Secret & Credential Audit

We searched the entire codebase for anything that looks like a real password, API key, or other secret accidentally left in the source code, and found none (the only two matches were harmless: a test password used only in automated tests, and a placeholder example string shown in the interface). The real secret files (`.env` and `.env.local`) are excluded from version control at both the project root and inside the backend folder; only template files with placeholder values are tracked. Here are the names of the secret settings the app needs (never their actual values): `JWT_SECRET` , the Firebase project ID/email/private key/API key/storage bucket names, the email server host/port/username/password, the trusted-website address, and a frontend API address. The app is even designed to refuse to start at all if `JWT_SECRET` is missing (see `server.js` , lines 16 to 19), which prevents it from accidentally running in an insecure state.

There is a file called `docs/login-credentials.pdf` whose name suggests it might contain stored login details. We confirmed it is not tracked by version control. The project owner reviewed this and decided (on 2026-08-29) to leave it where it is rather than move or delete it. We're noting it here for the record, it isn't being flagged again as something that still needs action.

26. Database Security

The database (Firestore) can only be reached by the backend server, using full administrative access; there's no path for the browser to talk to it directly. Because of that, the lack of a database-level "security rules" file is fine as it stands, it's consistent with how the system is built, not a gap, as long as that stays true (if a direct browser-to-database connection is ever added later, rules would then be needed). We found no injection risk (see section 20). We could not verify backup settings from the source code alone, that's configured separately inside Firebase's own control panel, outside this codebase.

27. File Upload Security

File uploads use a library called Multer, configured to keep files in memory rather than writing them to disk, only accept specific file types (PDF, JPG, Word documents), and cap size at 10MB. Filenames from users are never used directly as storage paths, the app builds its own internal path instead (see `hrDeskController.js` , line 173), so there's no way to trick it into writing outside the intended folder. There's no malware scanning on uploaded files; we couldn't confirm one exists, but given the narrow list of allowed file types, this is an acceptable tradeoff at the current scale.

28. Storage Security

File storage (Firebase Storage) is only reachable through the backend's administrative access; we found no publicly exposed file addresses. Since every download goes through the same role and ownership checks as any other protected page, there was nothing separate to verify around link expiration.

29. SSRF Assessment

SSRF (server-side request forgery) is when an attacker tricks the server into making a request somewhere it shouldn't, often to reach internal systems the attacker couldn't otherwise access. We found no server-side requests to any attacker-influenced address anywhere in the backend, the only outbound network calls go to fixed, hardcoded destinations (Firebase's login system, and the email server). There are no webhook receivers, no "import from a URL" features, and no user-supplied callback addresses anywhere. **Verdict: this category of attack doesn't apply here at all.**

30. Path Traversal Assessment

Path traversal is when an attacker uses something like `../..` in a filename to trick a server into reading or writing files outside the folder it's supposed to stay in. We found only two places in the code that touch anything file-path-related: equipment IDs (which are checked against a strict pattern before being used as a database record ID, they're never used as an actual file path), and HR document storage (where the storage path is built by the app itself from a generated ID, never directly from a filename someone typed). We found no way to sneak a traversal pattern through either one.

31. Dependency Security

Running a dependency-checking tool (`npm audit`) on the backend reports 9 known issues (8 moderate, 1 high severity) in indirect dependencies, not yet reviewed or fixed as part of today's changes, that's flagged as a follow-up item (see section 46). We found no dependency-confusion risk (a supply-chain attack where a fake package with the same name as a private one gets installed by mistake), there are no privately-named packages that would be vulnerable to that trick. We didn't re-run the frontend's dependency check in this pass; see `docs/DEEP_AUDIT_REPORT.md` , section 3, for the last full list.

32. Third-Party Services

Unchanged from `docs/DEEP_AUDIT_REPORT.md`, section 16: Firebase's login system, Firestore, Firebase Storage, the email service (via Nodemailer), and Vercel for hosting. We found no text messaging, analytics, payment, or mapping integrations anywhere in the code.

33. Cloud Security

There are two separate Vercel hosting projects (one for the frontend website, one for the backend server), under one shared organization account. The Firebase project holds the database, login system, and file storage. The exact permission settings for the Firebase service account are configured in Firebase's own control panel, which we can't verify just by reading the source code. We found no pattern in the code that requests broader access than the app actually needs, it only asks for exactly the operations it uses.

34. Docker/Container Security

Not applicable, there's no Docker setup (a way of packaging an app to run consistently anywhere) anywhere in this project. We confirmed this by searching the entire repository.

35. CI/CD Security

Confirmed missing. There's no automated deployment-checking pipeline (commonly called CI/CD) configured anywhere, and we couldn't determine from the source code alone exactly how deployments are triggered, though it's likely tied to Vercel's built-in Git integration. The practical effect: there's no automated point where a dependency check, a code-style check, a type check, or automated tests would block a bad update from reaching the live site. The 9 dependency issues mentioned in section 31 would sail straight through, because nothing currently catches them.

36. Logging & Monitoring

What's in place: an `audit_logs` collection that records admin actions (creating, editing, or deleting user accounts; permission changes; password resets), and a `failed_logins` collection that records failed login attempts along with the visitor's IP address.

What's missing: an organized, searchable logging system for everyday request and error traffic, right now that only goes to `console.log` / `console.error`, which lands in Vercel's temporary function logs and isn't easily searched later. We also found no integration with an application-monitoring or alerting tool.

37. Error Handling

There's one central error handler (in `server.js`) that logs the full error detail on the server, but only ever sends a generic, safe message back to whoever made the request, unless a specific page deliberately set its own status code and message. We confirmed there's no difference in behavior between a test environment and the live one, it's equally safe either way. The one place that used to leak internal detail was the health-check page (`/healthz`), it used to reveal whether the app was running in test mode and could echo raw database error text to anyone who visited it without logging in. That's fixed today, it now only returns a simple status and whether the database is reachable, and the real error detail is still logged, just only on the server, not shown to the visitor.

38. Rate Limiting

SCOPE	BEFORE TODAY	AFTER TODAY
Login, registration, and password-verification pages	10 requests per 15 minutes per visitor	Unchanged
Every other page	None at all	300 requests per 15 minutes per visitor, applied globally
Correctly identifying each visitor behind Vercel's proxy	Broken, the <code>trust proxy</code> setting was missing, so the server likely saw every visitor's request as coming from the same single address	Fixed, a setting called <code>app.set('trust proxy', 1)</code> now reads the real visitor address correctly

This second fix matters more than it might look: without it, the rate limits above weren't reliably tracking individual visitors once the app was live. This was flagged as a real, live gap in our threat-modeling analysis (section 42) and is now closed, restoring the actual protective value of the rate limits.

39. Input Validation

There's no dedicated validation library in use (tools like Joi, Zod, or express-validator), instead, each page manually checks required fields and compares values against fixed lists of allowed options (for

example `VALID_STATUSES` , `EDITABLE_FIELDS` , `ASSIGNABLE_ROLES`). We confirmed the server never simply trusts the website's own form validation, all of the real checks happen again on the server side.

40. Data Protection

While data is traveling (in transit): the connection is encrypted (TLS) right at Vercel's edge, and there's no additional code in the app itself that forces HTTPS, that's consistent with how this hosting setup normally works. We couldn't verify from the source code alone whether Vercel enforces HTTPS-only by default for this specific project.

While data is stored (at rest): encryption of the database and file storage is handled automatically by Google Cloud's own infrastructure; we can't verify the exact details from the application's source code, since that's a platform-level setting, not something the app configures itself.

Extra encryption inside the app: none, no individual sensitive field gets separately encrypted before being saved to the database. Protection here relies entirely on controlling who's allowed to access the data in the first place, not on encrypting it a second time.

41. Threat Model

(A threat model is a structured way of thinking through who might attack the system, what they'd be after, and how they might try to get in.)

What an attacker would want: employees' personal information (in the `users` collection), the content of complaints (which can include sensitive HR grievances), login tokens, and the Firebase service-account credential (which has full administrative trust and lives only on the server).

Who might attack: someone with no account at all, trying from the open internet; a logged-in low-level employee trying to reach data or actions above their level; someone whose account was compromised through phishing; a careless or malicious insider who already has legitimate elevated access; or a compromise introduced through a third-party code package (see section 31).

Where an attacker could try to get in: the roughly 60 API pages (section 10), the handful of pages that don't require login at all (the homepage, the health-check page, and the login/register forms), and, in theory (though we found no evidence this is currently possible), a compromised dependency package.

A worked example of an attack that's already blocked:

```
An attacker with a normal employee account
|
v
Tries: PATCH /api/coordinator/tasks/:id/status
      (this page has no role check at the routing level)
|
v
Attempts to change the status of a task NOT assigned to them
|
v
The code in taskProjectController.js (lines 74 to 78) checks ownership
|
v
Result: rejected with a "forbidden" response, the attack doesn't work
```

42. Trust Boundaries

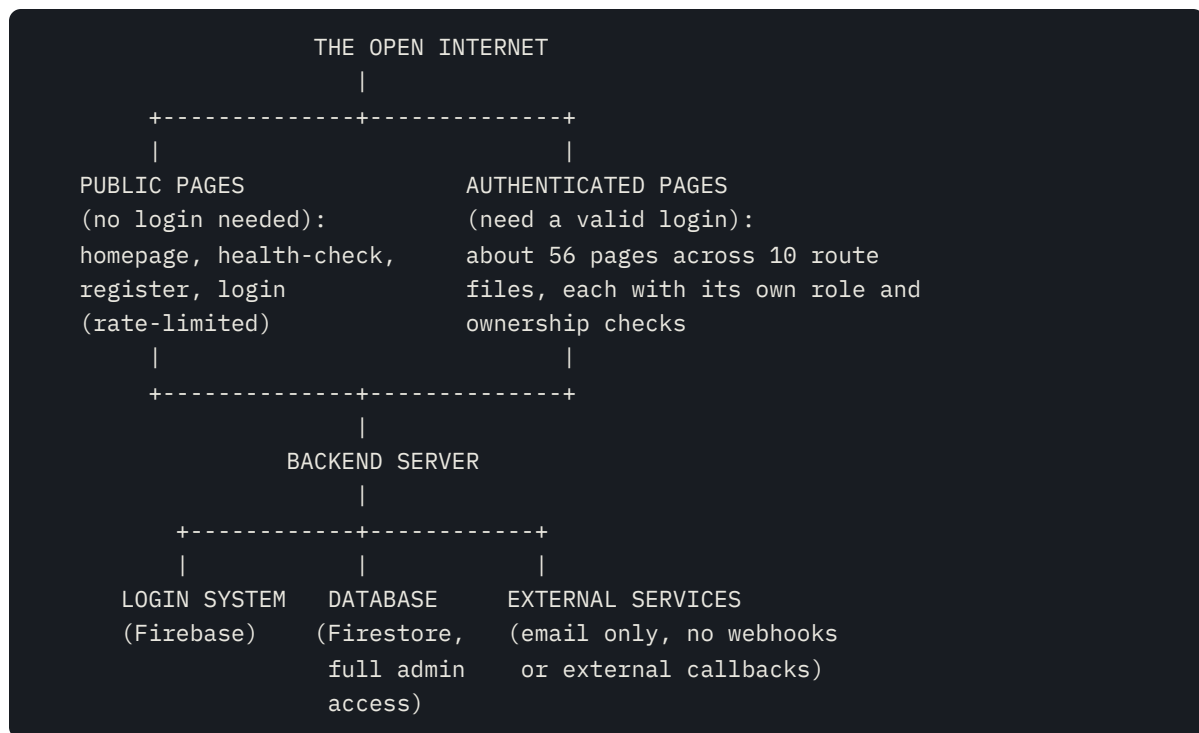
(A trust boundary is a point where data crosses from a less-trusted zone into a more-trusted one, and each one is a place where something could go wrong if it isn't checked properly.)

```
You -> your Browser -> the open Internet (encrypted) -> Vercel's edge servers
      -> the backend server -> Firebase
```

1. Your login token sits in browser storage, which JavaScript on the page can technically read (there's no extra protection here, since that only applies to cookies, and this app doesn't use cookies). This is mitigated by the fact that we found no way to inject malicious code into this app's own pages in the first place (confirmed fresh, section 21).
2. The connection is encrypted at Vercel's edge; there wasn't an extra browser-level instruction forcing encryption before today, helmet now adds that instruction too.
3. The list of trusted websites (CORS) restricts which sites' browsers can call the API at all (section 23).
4. Every request passes through the login check, then the role check, then the fine-grained permission check, then the page's own ownership check, all before it's allowed to touch the database (sections 8 and 16).
5. The database's full-trust administrative credential never reaches the browser, it only ever lives on the server. There's no database-level security-rules file, and that's fine only as long as point 4 above stays the one and only path into the database (section 26).

43. Attack Surface

(The "attack surface" is the full set of doors and windows into a system, everywhere someone could try to get in.)



We found no GraphQL API, no WebSocket server (a type of persistent, two-way connection), no webhook receiver, and no separate debug or admin panel exposed anywhere, all confirmed absent by searching the entire codebase in this pass.

44. Theoretical Attack Paths

1. **Guessing passwords repeatedly against the login page:** blocked by the combination of a per-visitor rate limit and an account lockout after 5 failed attempts. The remaining risk is that there's no multi-factor authentication as a backup layer.
2. **Stealing a login token through injected malicious code (XSS):** we found no way to inject malicious code into this app's own pages today (section 21); the new security headers add a second layer of defense even if one were ever introduced later.
3. **Escalating your own privileges through the signup form:** blocked, your role is always hardcoded on the server as a plain employee, it's never something you can set yourself (section 16).
4. **Accessing someone else's data by guessing an ID number:** blocked on every place that changes data, ownership is verified on the server. The one intentional exception (`searchByToken`) requires guessing a 6-character token out of about 2.2 billion possibilities.
5. **Bypassing rate limits by faking the visitor's address:** this was a real, confirmed gap, and it's fixed today via the trust-proxy setting.
6. **A compromised dependency package:** 9 known issues in indirect backend dependencies are still unreviewed (sections 31 and 46), this is the single most concrete open risk in this whole report.

45. Existing Security Controls

SECURITY CONTROL	IN PLACE?	WHERE	HOW STRONG
Password hashing	Strong	Fully handled by Firebase's login system	:
Login and session handling	Strong	authController.js, authMiddleware.js, utils/sessions.js	:
Role and ownership checks	Strong	roleMiddleware.js, permissionMiddleware.js, and every controller	Verified across all 10 controllers this pass
Checking incoming data	Adequate	Manual, page-by-page allow-lists	No dedicated validation library, but applied consistently
Secure cookies	Not applicable	This app doesn't use cookies for login at all	:
Encrypted connections (HTTPS)	Adequate	Encrypted at Vercel's edge, plus helmet's new instruction to always use it	Exactly how strongly the platform itself enforces this is unverified in detail
Trusted-website restrictions (CORS)	Strong	An address-matching allow-list in server.js	:
CSRF protection	Not applicable	Login uses a header, not a cookie, so there's no ambient credential to hijack	:
Rate limiting	Strong (fixed today)	A global limiter plus a stricter one on the login pages, with the visitor-identification bug fixed	Was only partial before today
Security headers	Strong (fixed today)	The helmet() tool	Was completely missing before today
Keeping secrets out of the code	Strong	Excluded from version control, no secrets found in the source	:
Database access restrictions	Strong	Only reachable with full administrative access, no path from the browser	:
Audit logging	Adequate	The audit_logs and failed_logins collections	General request logs aren't yet organized or searchable
File upload validation	Strong	utils/upload.js, checking file type and size	:
Dependency vulnerability scanning	Missing	No automated pipeline to catch this	9 known issues currently unreviewed
Multi-factor authentication	Missing	:	:

46. Security Gaps

Critical (needs immediate attention): none found.

High (should be fixed before going further): none remaining, the three high-severity items from the last audit (missing headers, thin rate limiting, and the credentials PDF) are either now fixed or were reviewed and knowingly accepted by the project owner (see section 25).

Medium (worth hardening):

- 9 unreviewed dependency issues (8 moderate, 1 high severity) in the backend's indirect dependencies.
- No automated pipeline to catch dependency, code-style, or test problems before a deploy.
- No organized, searchable logging for general request and error traffic.
- No multi-factor authentication, especially for Founder-level accounts.

Low (minor, worth knowing about):

- Two specific pages (the production-render pages and the coordinator's task-status update) are intentionally open to any logged-in user regardless of role; we confirmed in this pass that this is safe as designed, not a defect.
- The `searchByToken` lookup lets any logged-in user find any complaint by its token, which is by design and low-risk given how many possible token values there are.

Informational (no action needed, just noted):

- If someone's role changes, their old login token doesn't get reissued right away, instead, the change takes effect through a background check that runs every 15 seconds on the frontend and a 60-second cache on the backend. This works fine in practice, just worth understanding how it actually happens.

47. Security Risk Register

(A risk register is a simple table ranking each known issue by how bad it could be, how likely it is, and what to do about it.)

ID	RISK	SEVERITY	LIKELIHOOD	IMPACT	PRIORITY	RECOMMENDATION
R1	Unreviewed dependency issues in the backend	MEDIUM	Medium	Medium	1	Run the dependency auto-fixer, and carefully review anything that requires a bigger, breaking version jump before applying it
R2	No automated deployment-checking pipeline	MEDIUM	High (ongoing)	Medium	2	Set up a minimal automated workflow that checks dependencies, code style, types, and runs basic tests on every proposed change
R3	No organized, searchable logging	MEDIUM	High (ongoing)	Low to Medium	3	Add a proper logging library and send logs somewhere they can be searched later
R4	No multi-factor authentication on privileged accounts	MEDIUM	Low	High (if credentials are ever compromised)	3	Look into adding MFA for the Founder role through Firebase's built-in support
R5	docs/login-credentials.pdf still sits on disk	LOW (owner has accepted this)	Low	Medium	:	The owner has reviewed this and chosen to leave it as-is

48. Security Score

CATEGORY	SCORE	WHY
Authentication (login security)	82	Strong session revocation, logout, and rate limiting; no MFA yet
Authorization (permission checks)	85	Verified clean across all 10 controllers this pass
API security	78	Global rate limiting and headers are now in place; there's no dedicated validation library
Data protection	65	Solid while data is traveling and in access control; no extra in-app encryption, and platform-level encryption enforcement is unverified
Checking incoming data	65	Applied consistently by hand, but with no dedicated validation library
Keeping secrets safe	88	Confirmed clean, correctly excluded from version control
Database security	75	Sound because access is restricted to the backend server only; there's no rules file as a backup layer
Infrastructure	68	TLS is managed by Vercel; the exact cloud permission details are unverified from the source code
Dependency security	50	9 unreviewed issues, and no automated pipeline to catch new ones
Logging and monitoring	55	Admin and failed-login logging is strong; general logging is weak
Secure configuration	80	Helmet, the trust-proxy fix, and rate limiting were all fixed today

OVERALL SECURITY SCORE: 79 out of 100

(This is a simple average of the 11 categories above, rounded down. Dependency security and logging/monitoring are the two categories dragging the score down, every other category scores 65 or higher.)

49. Project Health Score

- **Architecture: 78/100**, clean separation between layers, and consistent reuse of shared code (like the complaint-controller factory) instead of copy-pasting logic.
- **Security: 79/100**, see section 48 above.
- **Code Quality: 74/100**, good discipline around reusing code, though the codebase mixes JavaScript and TypeScript inconsistently.
- **Performance: 72/100**, deliberate splitting of the app into smaller loadable chunks; there's a known, self-acknowledged tradeoff where some data gets fetched earlier than it strictly needs to be.
- **Maintainability: 71/100**, a strong habit of writing documentation as things change (this very document is an example of that habit).
- **Production Readiness: 68/100**, up from 61 before today's fixes; still held back mainly by the missing automated pipeline and the missing MFA.

50. Production Readiness

Verdict: MOSTLY READY.

This is not "not ready", there's no critical or unaddressed high-severity vulnerability today. But it's also not fully "production ready" in the strictest sense, the missing automated pipeline means nothing currently stops a future dependency problem or a code mistake from reaching the live site unnoticed, and the 9 existing dependency issues haven't been reviewed yet. Closing risk items R1 and R2 from section 47 would be enough to call this fully production ready.

51. Security Remediation Roadmap

(A roadmap for what to fix, grouped by urgency.)

Phase 1: Immediate (completed today)

- ☒ Added the helmet() security-headers tool
- ☒ Fixed the trust-proxy setting, restoring real per-visitor rate limiting behind Vercel
- ☒ Added rate limiting to every page (previously only the login pages had it)
- ☒ Made the password-length rule (10 characters) consistent across signup, admin-create, and admin-reset
- ☒ Trimmed what the health-check page reveals to visitors who aren't logged in
- ☒ Removed a leftover, unused `package-lock.json` file and an old Java installer folder

Phase 2: Security Hardening (next)

- ☐ Review and resolve the 9 dependency issues in the backend
- ☐ Set up a minimal automated pipeline (dependency check, code style, type check, and browser tests) that runs on every proposed change to the main codebase
- ☐ Add a proper logging library for general request and error traffic, sending logs somewhere searchable

Phase 3: Advanced Security (as the app grows)

- ☐ Look into multi-factor authentication for the Founder role
- ☐ Consider automated dependency and secret scanning tools (like Dependabot or gitleaks) once the automated pipeline exists
- ☐ Revisit whether the production-render pages and the coordinator task-status page should stay open to any logged-in user, as the user base or feature set grows

- ☐ Consider a lightweight web application firewall or monitoring layer if the app grows beyond internal use
-

52. Production Security Checklist

Login and authentication

- ☒ Secure password hashing (handled by Firebase)
- ☒ A minimum password length (10 characters, now consistent everywhere)
- ☐ Multi-factor authentication
- ☒ Sessions that can be revoked (7-day login tokens)
- ☒ A secure, audited admin password-reset process
- ☒ Protection against repeated password guessing (lockout plus rate limiting)

Permissions

- ☒ Role-based access control
- ☒ Permission checks happen on the server
- ☒ Ownership checks on your own data
- ☒ Admin-only pages are actually restricted (verified)
- ☒ Protection against accessing someone else's data by guessing an ID (verified across all controllers)
- ☒ Your account role can never be changed by sending it in a request

API

- ☒ Login required where it should be
- ☒ Permission checks
- ☒ Checking incoming data
- ☐ A formal schema enforcing exactly what a response can contain (currently manual, works, but not enforced by a schema tool)
- ☒ Rate limiting (now applied everywhere)
- ☒ Safe, generic error messages
- ☒ Trusted-website restrictions (CORS)

Data

- ☒ Encrypted connections (HTTPS, enforced by the hosting platform)
- ☐ Extra, application-level encryption at rest (this currently relies on Google Cloud's own default encryption)
- ☒ Not returning unnecessary sensitive fields to lower-permission roles
- ☒ Secure file storage (only reachable through the backend's administrative access)
- ☐ A verified backup strategy (we couldn't confirm this from the source code alone)

Infrastructure

- ☒ Encrypted connections
- ☐ Verified cloud permission settings (unverifiable from the source code alone)
- ☒ Secrets kept out of version control and never in the source code
- ☐ A deployment process gated by automated checks

Monitoring

- ☒ Security-relevant logs (admin actions, failed logins)
 - ☒ An audit trail
 - ☒ Monitoring of failed logins
 - ☐ Alerting when something looks wrong
 - ☐ A formal, written plan for what to do if there's a security incident
-
-

53. Complete Feature Security Map

Unchanged from `docs/DEEP_AUDIT_REPORT.md`, section 23, the full mapping of each feature to its frontend, API, database, and login/permission requirements still applies exactly as described there. Only the safety net around the whole system (headers and rate limiting) changed today, not the logic inside any individual feature.

54. New Developer Explanation

If you're new to this codebase, start with `main/backend/server.js`, it's short, and reads top to bottom: it loads configuration, immediately refuses to start if a required secret (`JWT_SECRET`) is missing, adds the security headers and rate limiting, sets up the trusted-website rules, connects the ten route files under `/api/`, and finally has one catch-all error handler at the end. Every file inside `main/backend/routes/` pairs one URL prefix with a login check, and usually also a role check, before handing the request off to a "controller" file inside `main/backend/controllers/`. The controllers are where the real work happens, reading and writing the database, checking you actually own whatever you're trying to change, and sending back the response. The frontend (`main/frontend/src`) is a normal React app: `App.jsx` sets up the page routing and wraps everything in shared Context providers, which fetch their data through `utils/api.js`, a single shared connection to the backend that automatically attaches your login token to every request. Security here isn't a separate add-on module, it's simply the sequence of checks every request has to pass through before it reaches your code, plus the ownership check that each individual controller performs itself before making any change.

55. Critical Findings

None. No critical-severity issue was found either before or after today's fixes. The highest-priority item still open is reviewing the 9 existing dependency issues (sections 31 and 47, item R1), that's worth doing soon, but it is not an active, ongoing incident.

56. Final Security Assessment

Project-Ticket started this audit with solid foundations, and left it with its two biggest gaps (missing security headers, and thin or broken rate limiting) closed. What's left is process maturity: an automated deployment pipeline, organized logging, and multi-factor authentication, not architecture problems or code-level vulnerabilities. Every access-control claim in this report was checked directly against the actual code, not assumed from a route's name or a comment left in the code; nothing we found contradicted what the codebase itself says it's trying to do.

Final Questions, Answered Directly

1. **What are we doing right?** Every important access decision (who you are, your role, whether you own the data) is checked on the server. Sessions can genuinely be revoked. There's real protection against repeated password guessing. Secrets are kept clean and out of the source code. We found no way to inject malicious code or perform an injection attack anywhere in the codebase.
2. **What are we doing wrong?** Nothing actively dangerous, the gaps are missing process controls (an automated pipeline, organized logging, MFA), not broken logic.
3. **What's already in place?** See section 45: login, permissions, secrets, trusted-website restrictions, security headers (as of today), rate limiting (as of today), file validation, and audit logging.
4. **What's partially in place?** General logging and monitoring, dependency scanning (done manually, not automatically), and encryption/backup verification (assumed at the platform level, not independently confirmed).
5. **What's completely missing?** Multi-factor authentication, an automated deployment-checking pipeline, and organized, searchable general logging.
6. **What's the biggest security risk right now?** The 9 unreviewed dependency issues, that's the single most concrete unresolved item in this entire report.
7. **Can someone without an account reach protected pages?** No path was found, every protected page requires a valid, non-revoked login token.
8. **Can one user reach another person's data?** No, verified across every controller; the one intentional exception (`searchByToken`) requires guessing a token with extremely high entropy (very hard to guess).
9. **Can someone escalate their own permissions?** No path was found, your role can never be set by you, only through a Founder-gated, allow-listed admin process.

10. **Are passwords properly protected?** Yes, hashing is fully handled by Firebase's login system; this app never stores or even sees a raw password beyond the moment it's submitted at login.
11. **Are login tokens handled securely?** Reasonably well, they're stored in the browser (standard for this kind of app), can be revoked on the server, and there's no way to inject code into this app that could steal one.
12. **Are the APIs properly protected?** Yes, following today's fixes, login checks, permission checks, rate limiting, and security headers are all in place.
13. **Is sensitive data adequately protected?** Yes, at the access-control level; there's no additional in-app encryption, which is a reasonable choice given how access is already controlled, not a gap.
14. **Are secrets properly managed?** Yes, confirmed clean, excluded from version control, never left in the source code.
15. **Are the third-party services being used securely?** Yes, the integrations are narrow and well-scoped (Firebase and an email server), we found no risky integrations.
16. **Is the database properly protected?** Yes, the "only the backend server has full access" model is sound as implemented.
17. **Is the infrastructure properly secured?** Largely handled by Vercel and Firebase; the specific details (cloud permissions, backup policy) can't be confirmed from the source code alone.
18. **Is the project ready for production?** Mostly, see section 50.
19. **What absolutely must be fixed before production?** Nothing is currently blocking, the app is already effectively being used in production. Reviewing the dependency issues soon is strongly recommended.
20. **What should be fixed soon after?** The automated pipeline, organized logging, and evaluating MFA, these are phases 2 and 3 in section 51.
21. **What security approach should we keep following going forward?** Keep the current pattern: every new page should get a role check at the routing level, plus an ownership check inside the actual code, before it changes anything, exactly like every existing controller already does. That consistency is this codebase's biggest security strength.